# File: backend_fastapi_mastery/02_pydantic_deep_mastery.md

# Pydantic Deep Mastery

## Why Pydantic Matters

Pydantic is the engine behind FastAPI's data validation, serialization, and documentation. Understanding it deeply is non-negotiable for production FastAPI work.

**What Pydantic does:**

- Validates incoming data against schemas
- Converts/coerces types automatically
- Serializes Python objects to JSON
- Generates JSON Schema (for OpenAPI docs)
- Provides clear error messages

**Core philosophy**: Define your data shape once, get validation, documentation, and serialization for free.

---

## Pydantic V1 vs V2

FastAPI supports both Pydantic V1 and V2. This guide covers V2 (current standard), with notes on V1 differences.

```
# Check your version
import pydantic
print(pydantic.VERSION)  # 2.x.x
```

**Key V2 changes:**

- Faster (5-50x in benchmarks)
- `model_validator` instead of `@root_validator`
- `field_validator` instead of `@validator`
- `model_dump()` instead of `.dict()`
- `model_dump_json()` instead of `.json()`
- `model_validate()` instead of `parse_obj()`

---

## Model Fundamentals

### Basic Model Definition

```python
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class User(BaseModel):
    id: int
    email: str
    name: str
    is_active: bool = True  # Default value
    created_at: Optional[datetime] = None  # Optional, defaults to None
```

### How Validation Works

```python
# Valid data
user = User(id=1, email="test@example.com", name="John")
print(user.id)  # 1 (int)

# Type coercion
user = User(id="123", email="test@example.com", name="John")
print(user.id)  # 123 (int) - string converted to int

# Missing required field
user = User(email="test@example.com")
# ValidationError: id field required

# Wrong type that can't be coerced
user = User(id="not-a-number", email="test@example.com", name="John")
# ValidationError: value is not a valid integer
```

### Validation Error Structure

```python
from pydantic import ValidationError

try:
    user = User(id="abc", email="not-an-email", name="")
except ValidationError as e:
    print(e.json())
```

Output:

```
[
  {
    "type": "int_parsing",
    "loc": ["id"],
    "msg": "Input should be a valid integer, unable to parse string as an
integer",
```

```
    "input": "abc"
  }
]
```

This structure is what FastAPI returns as 422 response bodies.

---

## Field Constraints

### Using Field()

```python
from pydantic import BaseModel, Field
from typing import Optional

class Product(BaseModel):
    # Required with constraints
    name: str = Field(
        ...,  # ... means required
        min_length=1,
        max_length=100,
        description="Product name",
        examples=["Widget Pro"]
    )

    # Numeric constraints
    price: float = Field(
        ...,
        gt=0,  # greater than
        le=10000,  # less than or equal
        description="Price in dollars"
    )

    # Optional with default
    stock: int = Field(
        default=0,
        ge=0,  # greater than or equal
        description="Available inventory"
    )

    # String patterns
    sku: str = Field(
        ...,
        pattern=r'^[A-Z]{3}-\d{4}$',
        description="SKU format: ABC-1234"
    )
```

### All Field Constraints

| Constraint | Applies To | Description |
|---|---|---|

| Constraint | Applies To | Description |
|---|---|---|
| `gt` | numbers | Greater than |
| `ge` | numbers | Greater than or equal |
| `lt` | numbers | Less than |
| `le` | numbers | Less than or equal |
| `multiple_of` | numbers | Must be multiple of value |
| `min_length` | strings, lists | Minimum length |
| `max_length` | strings, lists | Maximum length |
| `pattern` | strings | Regex pattern |
| `strict` | any | Disable type coercion |

### Strict Mode

```
from pydantic import BaseModel, Field

class StrictUser(BaseModel):
    # Normally "123" would be coerced to 123
    # With strict=True, it must be actual int
    id: int = Field(..., strict=True)

# This fails in strict mode
user = StrictUser(id="123")  # ValidationError

# Model-wide strict mode
class StrictModel(BaseModel):
    model_config = {"strict": True}

    id: int
    name: str
```

---

## Custom Validators

### Field Validators (V2)

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    email: str
    username: str
    password: str

    @field_validator('email')
```

```
    @classmethod
    def validate_email(cls, v: str) -> str:
        if '@' not in v:
            raise ValueError('Invalid email format')
        return v.lower()  # Normalize to lowercase

    @field_validator('username')
    @classmethod
    def validate_username(cls, v: str) -> str:
        if not v.isalnum():
            raise ValueError('Username must be alphanumeric')
        if len(v) < 3:
            raise ValueError('Username must be at least 3 characters')
        return v

    @field_validator('password')
    @classmethod
    def validate_password(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError('Password must be at least 8 characters')
        if not any(c.isupper() for c in v):
            raise ValueError('Password must contain uppercase letter')
        if not any(c.isdigit() for c in v):
            raise ValueError('Password must contain digit')
        return v
```

### Validator Modes

```
from pydantic import field_validator

class User(BaseModel):
    email: str

    # mode='before': Runs before Pydantic's validation
    @field_validator('email', mode='before')
    @classmethod
    def strip_whitespace(cls, v):
        if isinstance(v, str):
            return v.strip()
        return v

    # mode='after': Runs after Pydantic's validation (default)
    @field_validator('email', mode='after')
    @classmethod
    def normalize_email(cls, v: str) -> str:
        return v.lower()
```

### Model Validators (Cross-Field Validation)

```python
from pydantic import BaseModel, model_validator

class DateRange(BaseModel):
    start_date: datetime
    end_date: datetime

    @model_validator(mode='after')
    def validate_date_range(self) -> 'DateRange':
        if self.end_date <= self.start_date:
            raise ValueError('end_date must be after start_date')
        return self

class PasswordChange(BaseModel):
    current_password: str
    new_password: str
    confirm_password: str

    @model_validator(mode='after')
    def validate_passwords(self) -> 'PasswordChange':
        if self.new_password != self.confirm_password:
            raise ValueError('Passwords do not match')
        if self.new_password == self.current_password:
            raise ValueError('New password must be different')
        return self
```

Before Model Validators (Raw Input Processing)

```python
from pydantic import BaseModel, model_validator
from typing import Any

class FlexibleInput(BaseModel):
    items: list[str]

    @model_validator(mode='before')
    @classmethod
    def convert_single_to_list(cls, data: Any) -> Any:
        """Accept either single item or list"""
        if isinstance(data, dict):
            items = data.get('items')
            if isinstance(items, str):
                data['items'] = [items]
        return data

# Both work now
model = FlexibleInput(items=["a", "b"])
model = FlexibleInput(items="single")  # Converted to ["single"]
```

---

## Model Inheritance

### Basic Inheritance

```
from pydantic import BaseModel
from datetime import datetime
from typing import Optional

class TimestampMixin(BaseModel):
    created_at: datetime
    updated_at: Optional[datetime] = None

class UserBase(BaseModel):
    email: str
    name: str

class UserCreate(UserBase):
    password: str

class UserUpdate(BaseModel):
    email: Optional[str] = None
    name: Optional[str] = None

class UserInDB(UserBase, TimestampMixin):
    id: int
    hashed_password: str

class UserResponse(UserBase, TimestampMixin):
    id: int
    # Note: no password fields exposed
```

### The Create/Update/Response Pattern

```
# This is the standard pattern in FastAPI applications

class ItemBase(BaseModel):
    """Shared fields"""
    title: str
    description: Optional[str] = None
    price: float = Field(..., gt=0)

class ItemCreate(ItemBase):
    """Fields for creation"""
    # Inherits all from base
    # Add creation-specific fields if needed
    pass

class ItemUpdate(BaseModel):
    """Fields for update - all optional"""
    title: Optional[str] = None
    description: Optional[str] = None
    price: Optional[float] = Field(default=None, gt=0)
```

```python
class ItemResponse(ItemBase):
    """Fields returned to client"""
    id: int
    created_at: datetime

    model_config = {"from_attributes": True}  # For ORM conversion
```

### Generic Base Models

```python
from pydantic import BaseModel
from typing import Generic, TypeVar, List

T = TypeVar('T')

class PaginatedResponse(BaseModel, Generic[T]):
    items: List[T]
    total: int
    page: int
    page_size: int
    has_more: bool

# Usage
class UserResponse(BaseModel):
    id: int
    name: str

# PaginatedResponse[UserResponse] is a valid type
@app.get("/users", response_model=PaginatedResponse[UserResponse])
async def list_users(page: int = 1, page_size: int = 20):
    users, total = await user_repo.list(page, page_size)
    return PaginatedResponse(
        items=users,
        total=total,
        page=page,
        page_size=page_size,
        has_more=(page * page_size) < total
    )
```

---

## Serialization Control

### model_dump() Options

```python
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class User(BaseModel):
```

```
    id: int
    email: str
    name: str
    password_hash: Optional[str] = None
    created_at: datetime
    last_login: Optional[datetime] = None

user = User(
    id=1,
    email="test@example.com",
    name="John",
    password_hash="secret",
    created_at=datetime.now()
)

# Basic dump
user.model_dump()
# {'id': 1, 'email': 'test@example.com', 'name': 'John',
#  'password_hash': 'secret', 'created_at': datetime(...), 'last_login':
None}

# Exclude fields
user.model_dump(exclude={'password_hash'})

# Include only specific fields
user.model_dump(include={'id', 'email', 'name'})

# Exclude None values
user.model_dump(exclude_none=True)
# No 'last_login' in output

# Exclude fields that weren't explicitly set
user.model_dump(exclude_unset=True)
# No 'last_login' in output (wasn't set in constructor)

# By alias (if aliases defined)
user.model_dump(by_alias=True)
```

### Field Aliases

```
from pydantic import BaseModel, Field

class ExternalAPIResponse(BaseModel):
    # Python uses snake_case, but external API uses camelCase
    user_id: int = Field(..., alias='userId')
    first_name: str = Field(..., alias='firstName')
    last_name: str = Field(..., alias='lastName')

    model_config = {
        "populate_by_name": True  # Accept both alias and field name
    }
```

```python
# Parse from external API format
data = {"userId": 1, "firstName": "John", "lastName": "Doe"}
user = ExternalAPIResponse.model_validate(data)

# Or use Python names
user = ExternalAPIResponse(user_id=1, first_name="John", last_name="Doe")

# Serialize back to external format
user.model_dump(by_alias=True)
# {'userId': 1, 'firstName': 'John', 'lastName': 'Doe'}
```

### Custom Serializers

```python
from pydantic import BaseModel, field_serializer
from datetime import datetime
from decimal import Decimal

class Order(BaseModel):
    id: int
    amount: Decimal
    created_at: datetime
    status: str

    @field_serializer('amount')
    def serialize_amount(self, value: Decimal) -> str:
        """Serialize Decimal as string to preserve precision"""
        return str(value)

    @field_serializer('created_at')
    def serialize_datetime(self, value: datetime) -> str:
        """ISO format for dates"""
        return value.isoformat()

    @field_serializer('status')
    def serialize_status(self, value: str) -> str:
        """Uppercase status for external APIs"""
        return value.upper()
```

### Computed Fields

```python
from pydantic import BaseModel, computed_field
from datetime import datetime

class User(BaseModel):
    first_name: str
    last_name: str
    birth_date: datetime
```

```
    @computed_field
    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"

    @computed_field
    @property
    def age(self) -> int:
        today = datetime.now()
        return today.year - self.birth_date.year

user = User(first_name="John", last_name="Doe", birth_date=datetime(1990,
1, 1))
user.model_dump()
# {'first_name': 'John', 'last_name': 'Doe', 'birth_date': ...,
#  'full_name': 'John Doe', 'age': 34}
```

---

## Complex Types

### Nested Models

```
from pydantic import BaseModel
from typing import List, Optional

class Address(BaseModel):
    street: str
    city: str
    country: str
    postal_code: str

class ContactInfo(BaseModel):
    email: str
    phone: Optional[str] = None
    address: Optional[Address] = None

class Company(BaseModel):
    name: str
    contact: ContactInfo
    employees: List['Employee'] = []  # Forward reference

class Employee(BaseModel):
    id: int
    name: str
    company_id: int

# Validation cascades through nested models
company = Company(
    name="ACME",
    contact={
        "email": "contact@acme.com",
```

```
        "address": {
            "street": "123 Main St",
            "city": "New York",
            "country": "USA",
            "postal_code": "10001"
        }
    }
)
```

### Discriminated Unions

```
from pydantic import BaseModel, Field
from typing import Literal, Union
from typing_extensions import Annotated

class CreditCardPayment(BaseModel):
    type: Literal['credit_card']
    card_number: str
    expiry: str
    cvv: str

class BankTransferPayment(BaseModel):
    type: Literal['bank_transfer']
    account_number: str
    routing_number: str

class CryptoPayment(BaseModel):
    type: Literal['crypto']
    wallet_address: str
    currency: str

# Discriminated union - Pydantic uses 'type' field to determine which
model
Payment = Annotated[
    Union[CreditCardPayment, BankTransferPayment, CryptoPayment],
    Field(discriminator='type')
]

class Order(BaseModel):
    id: int
    amount: float
    payment: Payment

# Pydantic automatically validates against correct model
order = Order(
    id=1,
    amount=99.99,
    payment={"type": "credit_card", "card_number": "4111...", "expiry":
"12/25", "cvv": "123"}
)
print(type(order.payment))  # <class 'CreditCardPayment'>
```

### Recursive Models

```
from pydantic import BaseModel
from typing import List, Optional

class TreeNode(BaseModel):
    value: str
    children: List['TreeNode'] = []

# Self-referential structure
tree = TreeNode(
    value="root",
    children=[
        TreeNode(value="child1", children=[
            TreeNode(value="grandchild1")
        ]),
        TreeNode(value="child2")
    ]
)
```

### Constrained Collections

```
from pydantic import BaseModel, Field
from typing import List, Set, Dict

class Config(BaseModel):
    # List with length constraints
    tags: List[str] = Field(default=[], min_length=0, max_length=10)

    # Set (automatically dedupes)
    categories: Set[str] = Field(default=set())

    # Dict with constraints
    metadata: Dict[str, str] = Field(default={})

class Order(BaseModel):
    # Ensure at least one item
    items: List[str] = Field(..., min_length=1)
```

---

## Custom Types

### Annotated Types

```
from pydantic import BaseModel
from typing_extensions import Annotated
```

```python
from pydantic import AfterValidator, BeforeValidator
import re

def validate_phone(v: str) -> str:
    # Remove all non-digits
    digits = re.sub(r'\D', '', v)
    if len(digits) != 10:
        raise ValueError('Phone must be 10 digits')
    return f"({digits[:3]}) {digits[3:6]}-{digits[6:]}"

def validate_email_domain(v: str) -> str:
    if not v.endswith('@company.com'):
        raise ValueError('Must use company email')
    return v.lower()

PhoneNumber = Annotated[str, AfterValidator(validate_phone)]
CompanyEmail = Annotated[str, AfterValidator(validate_email_domain)]

class Employee(BaseModel):
    email: CompanyEmail
    phone: PhoneNumber

# These types are reusable across your codebase
employee = Employee(email="John@COMPANY.COM", phone="1234567890")
print(employee.email)  # john@company.com
print(employee.phone)  # (123) 456-7890
```

Custom Types with **get_pydantic_core_schema**

```python
from pydantic import GetCoreSchemaHandler
from pydantic_core import CoreSchema, core_schema
from typing import Any

class Money:
    """Custom money type that stores cents internally"""

    def __init__(self, cents: int):
        self.cents = cents

    @classmethod
    def from_dollars(cls, dollars: float) -> 'Money':
        return cls(int(dollars * 100))

    def to_dollars(self) -> float:
        return self.cents / 100

    def __repr__(self):
        return f"Money(${self.to_dollars():.2f})"

    @classmethod
    def __get_pydantic_core_schema__(
```

```
        cls, source_type: Any, handler: GetCoreSchemaHandler
    ) -> CoreSchema:
        return core_schema.no_info_after_validator_function(
            cls._validate,
            core_schema.union_schema([
                core_schema.int_schema(),
                core_schema.float_schema(),
            ]),

serialization=core_schema.plain_serializer_function_ser_schema(
                lambda m: m.cents,
                info_arg=False,
            ),
        )

    @classmethod
    def _validate(cls, value: int | float) -> 'Money':
        if isinstance(value, float):
            return cls.from_dollars(value)
        return cls(value)

class Order(BaseModel):
    amount: Money

order = Order(amount=99.99)
print(order.amount)  # Money($99.99)
print(order.model_dump())  # {'amount': 9999}
```

---

## ORM Integration

### From Attributes (ORM Mode)

```
from pydantic import BaseModel, ConfigDict
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

# SQLAlchemy model
class UserORM(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    email = Column(String)
    name = Column(String)
    hashed_password = Column(String)

# Pydantic model
class UserResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
    id: int
    email: str
    name: str
    # Note: hashed_password not included

# Convert ORM to Pydantic
db_user = UserORM(id=1, email="test@example.com", name="John",
hashed_password="...")
pydantic_user = UserResponse.model_validate(db_user)
```

### Handling Relationships

```
from pydantic import BaseModel, ConfigDict
from typing import List, Optional

# SQLAlchemy models
class CompanyORM(Base):
    __tablename__ = 'companies'
    id = Column(Integer, primary_key=True)
    name = Column(String)
    employees = relationship("EmployeeORM", back_populates="company")

class EmployeeORM(Base):
    __tablename__ = 'employees'
    id = Column(Integer, primary_key=True)
    name = Column(String)
    company_id = Column(Integer, ForeignKey('companies.id'))
    company = relationship("CompanyORM", back_populates="employees")

# Pydantic models
class EmployeeResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str

class CompanyResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    employees: List[EmployeeResponse] = []

# When you access company.employees, SQLAlchemy lazy loads them
# Pydantic will serialize the loaded relationships
```

---

## Validation Strategies

### Fail Fast vs Collect All Errors

```python
from pydantic import BaseModel, ValidationError

class User(BaseModel):
    email: str
    age: int
    name: str

# Default: Pydantic collects ALL errors
try:
    User(email=123, age="not-a-number", name=None)
except ValidationError as e:
    print(len(e.errors()))  # 3 errors collected
```

### Conditional Validation

```python
from pydantic import BaseModel, model_validator
from typing import Optional, Literal

class PaymentMethod(BaseModel):
    type: Literal['card', 'bank', 'crypto']

    # Card fields
    card_number: Optional[str] = None
    expiry: Optional[str] = None

    # Bank fields
    account_number: Optional[str] = None
    routing_number: Optional[str] = None

    # Crypto fields
    wallet_address: Optional[str] = None

    @model_validator(mode='after')
    def validate_fields_for_type(self) -> 'PaymentMethod':
        if self.type == 'card':
            if not self.card_number or not self.expiry:
                raise ValueError('Card payment requires card_number and
expiry')
        elif self.type == 'bank':
            if not self.account_number or not self.routing_number:
                raise ValueError('Bank payment requires account_number and
routing_number')
        elif self.type == 'crypto':
            if not self.wallet_address:
                raise ValueError('Crypto payment requires wallet_address')
        return self
```

### Validation Context

```python
from pydantic import BaseModel, field_validator, ValidationInfo

class Item(BaseModel):
    name: str
    price: float

    @field_validator('price')
    @classmethod
    def validate_price(cls, v: float, info: ValidationInfo) -> float:
        # Access validation context
        context = info.context or {}
        max_price = context.get('max_price', 10000)

        if v > max_price:
            raise ValueError(f'Price cannot exceed {max_price}')
        return v

# Pass context during validation
item = Item.model_validate(
    {'name': 'Widget', 'price': 500},
    context={'max_price': 100}
)  # ValidationError: Price cannot exceed 100
```

---

## Security Considerations

### Secret Fields

```python
from pydantic import BaseModel, SecretStr

class DatabaseConfig(BaseModel):
    host: str
    port: int
    username: str
    password: SecretStr  # Won't be printed/logged

config = DatabaseConfig(
    host="localhost",
    port=5432,
    username="admin",
    password="secret123"
)

print(config)
# host='localhost' port=5432 username='admin'
password=SecretStr('**********')

# Access the actual value when needed
actual_password = config.password.get_secret_value()
```

### Input Sanitization

```
from pydantic import BaseModel, field_validator
import html
import re

class Comment(BaseModel):
    content: str
    author: str

    @field_validator('content')
    @classmethod
    def sanitize_content(cls, v: str) -> str:
        # Escape HTML to prevent XSS
        return html.escape(v)

    @field_validator('author')
    @classmethod
    def sanitize_author(cls, v: str) -> str:
        # Remove potentially dangerous characters
        return re.sub(r'[<>"\'/]', '', v)
```

### Preventing Mass Assignment

```
from pydantic import BaseModel
from typing import Optional

class UserInDB(BaseModel):
    id: int
    email: str
    name: str
    is_admin: bool = False
    hashed_password: str

class UserUpdate(BaseModel):
    """Only allow updating these specific fields"""
    email: Optional[str] = None
    name: Optional[str] = None
    # is_admin NOT included - can't be updated via API

@app.patch("/users/{user_id}")
async def update_user(user_id: int, updates: UserUpdate):
    # Even if client sends {"is_admin": true}, it's ignored
    update_data = updates.model_dump(exclude_unset=True)
    # update_data will never contain is_admin
    await user_repo.update(user_id, update_data)
```

---

## Performance Optimization

### Model Caching

```
# Pydantic V2 automatically caches schema generation
# First validation of a model type is slower (schema building)
# Subsequent validations are fast

# Pre-build schemas at startup
from pydantic import TypeAdapter

UserListAdapter = TypeAdapter(List[User])

# Now list validation is optimized
users = UserListAdapter.validate_python(raw_data)
```

### Avoiding Repeated Validation

```
from pydantic import BaseModel

class Item(BaseModel):
    model_config = {"validate_assignment": False}  # Skip validation on
attribute assignment

    name: str
    price: float

item = Item(name="Widget", price=99.99)
item.price = -10  # No validation! Be careful

# Default behavior (validate_assignment=True in strict mode)
class StrictItem(BaseModel):
    model_config = {"validate_assignment": True}

    name: str
    price: float

strict_item = StrictItem(name="Widget", price=99.99)
strict_item.price = -10  # ValidationError if you have gt=0 constraint
```

### Lazy Validation

```
from pydantic import BaseModel

class HeavyModel(BaseModel):
    # Only validate when actually needed
    pass

# For large datasets, validate lazily
def process_items(raw_items: list):
```

```
    for raw in raw_items:
        try:
            item = HeavyModel.model_validate(raw)
            yield item
        except ValidationError:
            continue  # Skip invalid items
```

---

## Common Patterns

### Request/Response Symmetry

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

class ArticleBase(BaseModel):
    title: str = Field(..., min_length=1, max_length=200)
    content: str = Field(..., min_length=1)
    tags: list[str] = Field(default=[])

class ArticleCreate(ArticleBase):
    """What client sends to create"""
    pass

class ArticleUpdate(BaseModel):
    """What client sends to update (all optional)"""
    title: Optional[str] = Field(default=None, min_length=1,
max_length=200)
    content: Optional[str] = Field(default=None, min_length=1)
    tags: Optional[list[str]] = None

class ArticleResponse(ArticleBase):
    """What server returns"""
    id: int
    author_id: int
    created_at: datetime
    updated_at: Optional[datetime] = None

    model_config = {"from_attributes": True}
```

### Enum Validation

```
from pydantic import BaseModel
from enum import Enum

class OrderStatus(str, Enum):
    PENDING = "pending"
    PROCESSING = "processing"
```

```
    SHIPPED = "shipped"
    DELIVERED = "delivered"
    CANCELLED = "cancelled"

class Order(BaseModel):
    id: int
    status: OrderStatus

# Validates against enum values
order = Order(id=1, status="pending")  # Works
order = Order(id=1, status="invalid")  # ValidationError
```

### Partial Updates

```
from pydantic import BaseModel
from typing import Optional

class UserUpdate(BaseModel):
    email: Optional[str] = None
    name: Optional[str] = None
    bio: Optional[str] = None

@app.patch("/users/{user_id}")
async def update_user(user_id: int, updates: UserUpdate, db =
Depends(get_db)):
    # Only get fields that were actually sent
    update_data = updates.model_dump(exclude_unset=True)

    # {} if nothing sent
    # {"email": "new@example.com"} if only email sent
    # {"email": "new@example.com", "name": "New Name"} if both sent

    if not update_data:
        raise HTTPException(400, "No fields to update")

    await db.query(User).filter_by(id=user_id).update(update_data)
```

---

## Anti-Patterns to Avoid

### 1. Over-Validating in Routes

```
# WRONG: Manual validation when Pydantic does it
@app.post("/users")
async def create_user(user: UserCreate):
    if not user.email:  # Pydantic already validated this
        raise HTTPException(400, "Email required")
    if len(user.name) < 1:  # Pydantic already validated this
        raise HTTPException(400, "Name too short")
```

```python
# RIGHT: Trust Pydantic, add business logic validation in validators
class UserCreate(BaseModel):
    email: str = Field(..., min_length=1)
    name: str = Field(..., min_length=1)
```

### 2. Mutable Default Values

```python
# WRONG: Mutable default
class Config(BaseModel):
    items: list = []  # Shared across instances!

# RIGHT: Use Field with default_factory
class Config(BaseModel):
    items: list = Field(default_factory=list)
```

### 3. Exposing Internal Models

```python
# WRONG: Return ORM model directly
@app.get("/users/{user_id}")
async def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
    # Exposes password_hash!

# RIGHT: Use response model
@app.get("/users/{user_id}", response_model=UserResponse)
async def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter_by(id=user_id).first()
```

### 4. Not Using Inheritance

```python
# WRONG: Duplicating fields
class UserCreate(BaseModel):
    email: str
    name: str

class UserResponse(BaseModel):
    email: str  # Duplicated
    name: str   # Duplicated
    id: int
    created_at: datetime

# RIGHT: Use inheritance
class UserBase(BaseModel):
    email: str
    name: str
```

```python
class UserCreate(UserBase):
    password: str

class UserResponse(UserBase):
    id: int
    created_at: datetime
```

---

## Mastery Checkpoints

### Conceptual Questions

1. **What's the difference between `exclude_unset` and `exclude_none` in `model_dump()`?**

   *Answer*: `exclude_unset` excludes fields that weren't explicitly set during model creation (even if they have a default). `exclude_none` excludes fields whose value is `None`. Example: if field has `default=None` and you don't set it, `exclude_unset=True` excludes it (wasn't set), but `exclude_none=True` also excludes it (value is None). If you set it explicitly to `None`, `exclude_unset` includes it, `exclude_none` excludes it.

2. **When would you use `mode='before'` vs `mode='after'` in a field validator?**

   *Answer*: `mode='before'` runs before Pydantic's type validation - use it when you need to transform raw input (strip whitespace, handle multiple input formats, parse custom strings). `mode='after'` runs after type validation - use it when you want to work with properly typed values and add business logic validation. `before` sees raw input (could be any type), `after` sees typed values.

3. **How does `from_attributes=True` work with lazy-loaded ORM relationships?**

   *Answer*: When Pydantic validates an ORM model with `from_attributes=True`, it accesses attributes on the ORM object. For lazy-loaded relationships, this triggers the lazy load. If the session is closed, you get a DetachedInstanceError. Solutions: use eager loading (`joinedload`), ensure session is open during serialization, or don't include relationships in response model.

4. **What's the purpose of discriminated unions?**

   *Answer*: Discriminated unions use a specific field (discriminator) to determine which model in a Union to validate against. This is more efficient than trying each model until one works, and provides clearer error messages. It's ideal for polymorphic data where a type field indicates the variant. Example: payment methods with type="card" vs type="bank".

5. **How do you handle circular references in Pydantic models?**

   *Answer*: Use forward references (string annotations) and call `model_rebuild()` after all models are defined. Example: `children: List['TreeNode']`. In Python 3.10+, you can use `from __future__ import annotations` to make all annotations string-based by default.

### Scenario Questions

6. **Design Pydantic models for an e-commerce cart system with products, variants, and quantities.**

   *Answer*:

```
class ProductVariant(BaseModel):
    id: int
    sku: str
    attributes: dict[str, str]  # {"color": "red", "size": "M"}
    price: Decimal
    stock: int

class CartItem(BaseModel):
    variant_id: int
    quantity: int = Field(..., ge=1, le=100)

    @field_validator('quantity')
    @classmethod
    def validate_quantity(cls, v):
        if v < 1:
            raise ValueError('Quantity must be at least 1')
        return v

class Cart(BaseModel):
    items: list[CartItem] = Field(default=[], max_length=50)

    @model_validator(mode='after')
    def validate_unique_variants(self) -> 'Cart':
        variant_ids = [item.variant_id for item in self.items]
        if len(variant_ids) != len(set(variant_ids)):
            raise ValueError('Duplicate variants in cart')
        return self
```

7. **You receive JSON from an external API with inconsistent field naming (mix of camelCase and snake_case). How do you handle it?**

   *Answer*:

```
from pydantic import BaseModel, Field, model_validator

class ExternalData(BaseModel):
    user_id: int = Field(alias='userId')
    first_name: str = Field(alias='firstName')
    # Handle both conventions
    last_name: str = Field(alias='lastName')

    model_config = {"populate_by_name": True}

    @model_validator(mode='before')
    @classmethod
```

```
    def normalize_keys(cls, data):
        # Convert any snake_case to camelCase for consistent aliasing
        normalized = {}
        for key, value in data.items():
            # If already camelCase, use as-is
            if key in ['userId', 'firstName', 'lastName']:
                normalized[key] = value
            # Convert snake_case
            elif '_' in key:
                camel = ''.join(
                    word.capitalize() if i else word
                    for i, word in enumerate(key.split('_'))
                )
                normalized[camel] = value
            else:
                normalized[key] = value
        return normalized
```

8. **How would you validate that a password meets complexity requirements and matches a confirmation field?**

   *Answer*:

```
class PasswordChange(BaseModel):
    current_password: str
    new_password: str = Field(..., min_length=8)
    confirm_password: str

    @field_validator('new_password')
    @classmethod
    def validate_password_complexity(cls, v):
        if not any(c.isupper() for c in v):
            raise ValueError('Must contain uppercase letter')
        if not any(c.islower() for c in v):
            raise ValueError('Must contain lowercase letter')
        if not any(c.isdigit() for c in v):
            raise ValueError('Must contain digit')
        if not any(c in '!@#$%^&*' for c in v):
            raise ValueError('Must contain special character')
        return v

    @model_validator(mode='after')
    def validate_passwords_match(self) -> 'PasswordChange':
        if self.new_password != self.confirm_password:
            raise ValueError('Passwords do not match')
        if self.new_password == self.current_password:
            raise ValueError('New password must be different')
        return self
```

9. **Design a model that accepts either a file URL or base64-encoded file content.**

*Answer*:

```
from pydantic import BaseModel, model_validator
from typing import Optional
import base64

class FileInput(BaseModel):
    url: Optional[str] = None
    base64_content: Optional[str] = None
    filename: str

    @model_validator(mode='after')
    def validate_source(self) -> 'FileInput':
        if self.url and self.base64_content:
            raise ValueError('Provide either url or base64_content,
not both')
        if not self.url and not self.base64_content:
            raise ValueError('Must provide url or base64_content')
        return self

    @field_validator('base64_content')
    @classmethod
    def validate_base64(cls, v):
        if v is None:
            return v
        try:
            base64.b64decode(v)
        except Exception:
            raise ValueError('Invalid base64 encoding')
        return v

    def get_content(self) -> bytes:
        if self.base64_content:
            return base64.b64decode(self.base64_content)
        # Fetch from URL
        raise NotImplementedError("URL fetching")
```

10. **How do you handle API versioning where v1 and v2 have different response shapes?**

*Answer*:

```
class UserV1(BaseModel):
    id: int
    name: str
    email: str

class UserV2(BaseModel):
    id: int
    profile: dict  # Nested structure in v2
    contact: dict
```

```
    @classmethod
    def from_user(cls, user: UserORM) -> 'UserV2':
        return cls(
            id=user.id,
            profile={"name": user.name, "bio": user.bio},
            contact={"email": user.email, "phone": user.phone}
        )

@app.get("/users/{user_id}")
async def get_user(
    user_id: int,
    version: str = Header(default="1", alias="API-Version")
):
    user = await user_repo.find(user_id)

    if version == "2":
        return UserV2.from_user(user)
    return UserV1.model_validate(user)
```

---

## Interview Framing

When discussing Pydantic in interviews:

1. **Emphasize the single source of truth**: "I define data shapes as Pydantic models, and that single definition drives validation, serialization, and documentation. Changes propagate automatically."

2. **Show security awareness**: "I always use explicit response models to prevent accidentally exposing sensitive fields. The model acts as a whitelist of what can leave the API."

3. **Discuss performance understanding**: "Pydantic V2 is significantly faster due to Rust core. For hot paths, I pre-build TypeAdapters. For large payloads, I use streaming validation."

4. **Demonstrate practical patterns**: "I use the Base/Create/Update/Response model hierarchy to share validation while controlling what's exposed. It scales well as the API grows."

5. **Connect to system design**: "Pydantic models aren't just for HTTP—I use them for configuration, queue messages, and cache serialization. Consistent validation across all boundaries."